



Tec Gurus
SOMOS Y FORMAMOS EXPERTOS EN T.I.

CURSO

Linux Desde Cero

CLASE 1

Fundamentos, archivos y permisos

www.tecgurus.net

INSTRUCTOR

Ing. Abimael Domínguez



Clase 1 — Fundamentos, archivos y permisos

Archivos:

01-introduccion-a-linux.md (capítulo 1) · 03-estructura-del-sistema-de-archivos-de-linux.md (capítulo 3) · 07-el-shell.md (secciones 7.1–7.15).

HORA	ACTIVIDAD
09:00–09:30	Diagnóstico y modelo mental: usuario, host, kernel, distribución, terminal y shell.
09:30–10:10	Usuarios, grupos y <code>sudo</code> ; lectura de evidencia antes de modificar cuentas.
10:10–10:45	Consulta de paquetes, ayuda y práctica controlada con <code>apt</code> .
10:45–11:00	Checkpoint, preguntas y repetición; sin contenido nuevo.
11:00–11:30	Receso.
11:30–12:00	Rutas, jerarquía y tipos de archivo; observar antes de actuar.
12:00–12:35	<code>pwd</code> , <code>ls</code> , <code>cd</code> , <code>mkdir</code> y <code>touch</code> , con práctica por comando.
12:35–13:05	<code>cp</code> , <code>mv</code> , <code>file</code> y enlaces: origen, destino y resultado esperado.
13:05–13:35	Permisos y práctica guiada: configuración, script y enlace.
13:35–13:52	Reto corto, clínica de errores y evidencia.
13:52–14:00	Exit ticket, semáforo final y cierre.

No recortar: navegación, operaciones y permisos.

Recortar primero: comparación de distribuciones y detalles de cuentas.

Índice interactivo

Selecciona un apartado para ir directamente a su contenido. En cada encabezado encontrarás el enlace para volver aquí.

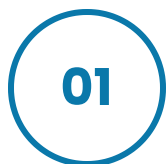
CAPÍTULO 1	
Introducción a Linux	
Índice	→
Objetivos	→
Antes de empezar	→
Ruta práctica de la Clase 1	→
1.1 ¿Qué es Linux?	→
Comprobar el sistema	→
1.2 ¿Qué son las distribuciones?	→
Administración esencial de paquetes	→
Ayuda antes de ejecutar	→
1.3 Entorno de trabajo: shell y entorno gráfico	→
1.4 Usuarios y grupos	→
¿Qué hace getent?	→
Crear un usuario de laboratorio	→
Sintaxis general	→
Ejemplo resuelto para copiar	→
Práctica guiada resuelta	→
Errores frecuentes	→
Reto 1 — Inventario reproducible	→
Criterios de comprobación	→
Checklist	→

Índice interactivo

Sistema de archivos y comandos esenciales del shell.

CAPÍTULO 3	
Estructura del sistema de archivos	
Índice	→
Objetivos	→
Antes de empezar	→
3.1 Tipos de archivo	→
3.2 Enlaces	→
• Modelo mental: nombre, inode y contenido	→
• Ejemplo guiado	→
• Comprueba qué se rompe y qué permanece	→
3.3 Rutas o paths	→
3.4 Jerarquía principal	→
3.5 Acceso a sistemas de archivos	→
• Antes de continuar	→
• Modelo mental	→
• Sintaxis y ejemplo guiado	→
3.6 Permisos	→
Práctica guiada resuelta	→
Errores frecuentes	→
Reto 3 — Directorio compartido	→
• Criterios de comprobación	→
Checklist	→

CAPÍTULO 7.1–7.15	
El shell	
Índice	→
Objetivos	→
Antes de empezar	→
7.1–7.3 Shell, comandos y directorio personal	→
7.4 Listar: ls	→
7.5–7.8 Directorios y navegación	→
7.9 Unidades y montajes	→
7.10 Copiar: cp	→
7.11 Mover y renombrar: mv	→
7.12 Enlaces: ln	→
7.13 Borrar: rm	→
7.14 Identificar: file y stat	→
7.15 Permisos y propiedad	→
Antes de continuar con la Clase 2	→
Ruta práctica de la Clase 2	→



FUNDAMENTOS

Introducción a Linux

Kernel, distribución, terminal, shell, paquetes, usuarios, grupos y privilegios administrativos.

Capítulo 1

Ubuntu 24.04

apt

sudo

Índice

[Índice ↑](#)

- [Objetivos](#)
- [Antes de empezar](#)
- [Ruta práctica de la Clase 1](#)
- [1.1 ¿Qué es Linux?](#)
- [1.2 ¿Qué son las distribuciones?](#)
- [1.3 Entorno de trabajo: shell y entorno gráfico](#)
- [1.4 Usuarios y grupos](#)
- [Práctica guiada resuelta](#)
- [Errores frecuentes](#)
- [Reto 1](#)

Objetivos

[Índice ↑](#)

- Distinguir kernel, distribución, shell, terminal y escritorio.
- Identificar la distribución y el kernel de una máquina.
- Consultar ayuda e instalar paquetes en Ubuntu.
- Trabajar con usuarios, grupos y privilegios mediante `sudo`.

Antes de empezar

[Índice ↑](#)

Abre una terminal y sitúate en la raíz del curso:

```
cd ~/linux-desde-cero
pwd
ls README.md
```

Si `ls README.md` indica que no encuentra el archivo, revisa dónde clonaste el repositorio antes de continuar. En este capítulo instalarás o crearás temporalmente:

- el paquete `tree`, para visualizar directorios;
- un grupo llamado `devops-lab`;
- un usuario de práctica llamado `alumno`;
- la carpeta `~/curso-linux/evidencias` para guardar resultados.

ANTES DE EJECUTAR.

Esta es la primera sesión. `~/curso-linux` y `laboratorio/` son espacios de práctica; puedes crearlos. No crees archivos vacíos dentro de `data/` para ocultar un error de ruta.

Ruta práctica de la Clase 1

[Índice ↑](#)

1. Identifica qué sistema y usuario estás usando.
2. Comprende qué cambia `sudo` y qué cambia un gestor de paquetes.
3. Aprende a recorrer y reconocer rutas antes de modificarlas.
4. Crea una estructura segura, copia y renombra archivos.
5. Comprueba tipo, enlace, dueño y permisos antes de corregirlos.

1.1 ¿Qué es Linux?

[Índice ↑](#)

SITUACIÓN REAL.

Una alerta puede mencionar “Linux”, “Ubuntu” o “kernel”. Para operar un servidor con seguridad debes saber si hablan del núcleo que controla recursos o de la distribución que instala y configura herramientas.

Linux es el **kernel**: administra CPU, memoria, dispositivos, procesos y sistemas de archivos. Una distribución combina ese kernel con herramientas, bibliotecas, un gestor de paquetes y decisiones de configuración.

SALIDA REPRESENTATIVA

hardware → kernel Linux → herramientas del sistema → aplicaciones → usuario

Comprobar el sistema

[Índice ↑](#)

TERMINAL · BASH

```
uname -r  
cat /etc/os-release
```

Las opciones usadas:

- `uname -r`: `-r` pide el *release* o versión del kernel en ejecución.
- `cat /etc/os-release`: no tiene bandera; lee el archivo estándar que describe la distribución.

SALIDA REPRESENTATIVA

SALIDA REPRESENTATIVA

```
6.8.0-xx-generic  
PRETTY_NAME="Ubuntu 24.04.x LTS"  
ID=ubuntu  
VERSION_ID="24.04"
```

- `uname -r`: muestra la versión del kernel en ejecución.
- `/etc/os-release`: describe la distribución, no el kernel.
- `-r`: solicita el *kernel release*.

COMPRUEBA.

La primera salida identifica el kernel; las líneas `Pretty_Name` e `ID` identifican la distribución. No tienen por qué usar el mismo número de versión.

1.2 ¿Qué son las distribuciones?

[Índice ↑](#)

SITUACIÓN REAL.

Un runbook puede decir `apt install`, pero otro servidor usar `dnf`. El objetivo es reconocer el flujo consultar → revisar → instalar, no aprender una lista aislada de comandos.

Las distribuciones comparten el kernel, pero cambian herramientas, versiones, soporte y paquetes.

Familia	Ejemplos	Gestor habitual
Debian	Ubuntu, Debian	<code>apt</code> / <code>dpkg</code>
RHEL	Red Hat Enterprise Linux, Fedora, Rocky Linux	<code>dnf</code> / <code>rpm</code>
Arch	Arch Linux, Manjaro	<code>pacman</code>

El laboratorio usa Ubuntu. Aprender los conceptos y consultar ayuda es más importante que memorizar tres gestores.

Administración esencial de paquetes

[Índice ↑](#)

```
sudo apt update
apt search tree
apt show tree
sudo apt install tree
tree --version
```

Las opciones y subcomandos usados:

- `sudo`: ejecuta sólo el comando que sigue con privilegios administrativos.
- `apt update`: actualiza el catálogo local de paquetes; no actualiza programas instalados.
- `apt search <texto>`: busca por nombre o descripción.
- `apt show <paquete>`: muestra versión, tamaño, dependencias y descripción.
- `apt install <paquete>`: instala el paquete indicado después de revisarlo.
- `tree --version`: `--version` confirma qué versión se instaló.
- `apt update`: actualiza el índice local; no actualiza todavía los programas.
- `search`: busca por nombre y descripción.
- `show`: enseña versión, tamaño y dependencias.
- `install`: instala el paquete solicitado.

- `sudo` : ejecuta sólo ese comando con privilegios administrativos.

En este capítulo **no eliminamos `tree` todavía**, porque se utiliza en la práctica final. Cuando termines y sólo si deseas retirarlo, la sintaxis es:

```
sudo apt remove tree
```

En una distribución con DNF, el flujo equivalente usa `dnf search`, `dnf info`, `dnf install` y `dnf remove`.

Ayuda antes de ejecutar

[Índice ↑](#)

```
man uname
uname --help
type uname
command -v uname
```

- `man` : manual completo; sal con `q`.
- `--help` : resumen rápido de opciones.
- `type` : indica si el nombre es alias, builtin o ejecutable.
- `command -v` : muestra cómo lo resolverá el shell.

CUIDADO.

`apt upgrade` cambia paquetes ya instalados y no forma parte de esta práctica. Primero aprende a consultar qué haría una acción administrativa.

1.3 Entorno de trabajo: shell y entorno gráfico

SITUACIÓN REAL.

La terminal es la ventana de texto; Bash es el intérprete que entiende lo que escribes. En una EC2 puedes tener shell sin escritorio, y en GNOME/KDE puedes abrir distintas terminales que usen el mismo shell.

- **Terminal:** interfaz que recibe texto y muestra resultados.
- **Shell:** programa que interpreta comandos; Bash es el principal del curso.
- **Escritorio:** entorno gráfico como GNOME o KDE Plasma.
- **Servidor gráfico/compositor:** conecta aplicaciones gráficas, entrada y pantalla.

```
echo "$SHELL"
ps -p "$$" -o comm=
tty
```

SALIDA REPRESENTATIVA

```
/bin/bash
bash
/dev/pts/0
```

- `$SHELL` contiene el shell configurado para el usuario.
- `$$` es el PID del shell actual.
- `tty` identifica la terminal asociada a la sesión.

Las piezas que debes reconocer:

- `$SHELL`: variable con el shell configurado para tu usuario.
- `$$`: identificador del shell actual; lo usaremos sólo para observar, no para terminar procesos.
- `ps -p <PID> -o comm=`: `-p` selecciona un proceso y `-o` elige la columna mostrada.

1.4 Usuarios y grupos

[Índice ↑](#)

SITUACIÓN REAL.

Cuando un archivo rechaza acceso, la pregunta no es “¿cómo obtengo permisos totales?”, sino “¿con qué usuario trabajo, a qué grupos pertenezco y qué autorización requiere la tarea?”.

Linux separa identidades y permisos mediante UID y GID.

TERMINAL · BASH

```
whoami
id
groups
getent passwd "$USER"
```

SALIDA REPRESENTATIVA

SALIDA REPRESENTATIVA

```
uid=1000(ubuntu) gid=1000(ubuntu) groups=1000(ubuntu),27(sudo)
```

- `uid` : identidad numérica del usuario.
- `gid` : grupo principal.
- `groups` : grupos adicionales; `sudo` suele conceder administración en Ubuntu.

Las opciones y consultas usadas:

- `whoami` : muestra el nombre efectivo del usuario actual.
- `id` : muestra UID, GID y grupos en una sola salida.
- `groups` : lista grupos por nombre.
- `getent passwd "$USER"` : consulta el registro de tu usuario; `passwd` es la base de cuentas y `$USER` se sustituye por tu nombre de sesión.

¿Qué hace `getent` ?

[Índice ↑](#)

`getent` significa **get entries**: obtener registros de las bases de datos que Linux tiene configuradas en `/etc/nsswitch.conf`.

Esto es importante porque los usuarios y grupos no siempre provienen únicamente de `/etc/passwd` y `/etc/group`. En una organización también pueden venir de LDAP, Active Directory u otro servicio de identidades. `getent` consulta esas fuentes mediante la configuración del sistema y presenta una salida uniforme.

SINTAXIS GENERAL

TERMINAL · BASH

```
getent <base_de_datos> [clave]
```

EJEMPLOS RESUELTOS

TERMINAL · BASH

```
getent passwd "$USER"  
getent passwd root  
getent group sudo
```

- `passwd` : base de datos de cuentas de usuario.
- `group` : base de datos de grupos.
- `"$USER"` : se sustituye automáticamente por tu usuario actual.
- `root` y `sudo` : claves concretas que queremos buscar.

Una salida de usuario tiene siete campos separados por :

SALIDA REPRESENTATIVA

```

| ubuntu:x:1000:1000:Ubuntu:/home/ubuntu:/bin/bash
| | | | | |
| | | | | └─ shell de inicio
| | | | └─ directorio personal
| | | └─ descripción de la cuenta
| | └─ GID del grupo principal
| └─ UID del usuario
└─ la contraseña/hash no se guarda aquí
└─ nombre de usuario

```

La `x` no es la contraseña. Indica que la información protegida se guarda en `/etc/shadow`, que sólo puede leer `root`.

Para esta primera clase basta ejecutar `getent passwd alumno` y leer la salida. Las condiciones, redirecciones y estados de salida se automatizan en la Clase 3.

Crear un usuario de laboratorio

[Índice ↑](#)

CUIDADO.

Crear usuarios y grupos cambia el equipo. La práctica se realiza sólo en una VM o instancia de laboratorio autorizada; si trabajas en un equipo corporativo, observa la demostración y no copies el bloque.

Vamos a crear dos objetos diferentes:

Objeto	Nombre usado	Propósito
Grupo	devops-lab	Representa a un equipo que comparte permisos.
Usuario	alumno	Cuenta temporal con la que practicaremos administración.

El nombre del usuario es `alumno`; aparece como último argumento en comandos como `useradd`, `passwd`, `id` y `usermod`.

Sintaxis general

[Índice ↑](#)

```
sudo groupadd <nombre_grupo>
sudo useradd -U -m -s /bin/bash -G <nombre_grupo> <nombre_usuario>
sudo passwd <nombre_usuario>
sudo usermod -aG sudo <nombre_usuario>
id <nombre_usuario>
```

- `<nombre_grupo>`: grupo adicional al que pertenecerá la cuenta.
- `<nombre_usuario>`: nombre de la cuenta que quieres crear.
- Los marcadores `<...>` se sustituyen; no se escriben literalmente.

Ejemplo resuelto para copiar

[Índice ↑](#)

```
sudo groupadd devops-lab
sudo useradd -U -m -s /bin/bash -G devops-lab alumno
sudo passwd alumno
sudo usermod -aG sudo alumno
id alumno
```

Qué ocurre en cada línea:

1. `groupadd devops-lab` crea el grupo `devops-lab`.
 2. `useradd ... alumno` crea el usuario `alumno`.
 3. `passwd alumno` solicita dos veces una contraseña para `alumno`; mientras escribes no se muestran caracteres, pero el teclado sí está funcionando.
 4. `usermod -aG sudo alumno` agrega `alumno` al grupo administrativo `sudo`.
 5. `id alumno` confirma UID, grupo principal y grupos secundarios después del cambio.
- `-m`: crea el directorio personal.
 - `-U`: crea un grupo principal con el mismo nombre del usuario.
 - `-s`: define el shell de inicio.
 - `-G`: asigna grupos secundarios.

- `-aG`: **añade** grupos; omitir `-a` puede reemplazar membresías existentes.

Comprueba que la cuenta y su home existen:

```
getent passwd alumno
id alumno
sudo ls -ld /home/alumno
```

SALIDA REPRESENTATIVA

```
alumno:x:1001:1001::/home/alumno:/bin/bash
uid=1001(alumno) gid=1001(alumno) groups=1001(alumno),27(sudo),1002(devops-lab)
drwxr-x--- ... alumno alumno ... /home/alumno
```

Los números UID/GID pueden ser distintos. Lo importante es ver `alumno`, `/home/alumno`, `/bin/bash`, `sudo` y `devops-lab`. La nueva membresía de grupo se aplica al iniciar una sesión nueva; no necesitas cambiar de usuario para continuar el capítulo.

Limpieza, después de la práctica:

```
sudo userdel -r alumno
sudo groupdel devops-lab
```

`userdel -r alumno` elimina la cuenta y `/home/alumno`. Ejecútalo únicamente cuando ya no necesites los archivos de ese usuario. Después se elimina el grupo temporal `devops-lab`.

Práctica guiada resuelta

[Índice ↑](#)

SITUACIÓN REAL.

Antes de administrar un equipo necesitas dejar una evidencia que otra persona pueda leer. En esta primera clase la evidencia se guarda con Text Editor; en la Clase 3 aprenderás a generarla automáticamente desde la terminal.

Objetivo: inspeccionar el host, instalar una herramienta y organizar una evidencia.

Esta práctica se ejecuta con tu usuario habitual, no con la cuenta `alumno`. Creará `~/curso-linux/evidencias/` y después mostrará la estructura con `tree`.

TERMINAL · BASH

```
mkdir -p ~/curso-linux/evidencias
sudo apt update
sudo apt install -y tree
tree ~/curso-linux
```

Las opciones usadas:

- `mkdir -p`: crea las carpetas padre faltantes y no falla si ya existen.
- `apt install -y`: `-y` confirma automáticamente; úsalo sólo después de revisar el paquete con `apt show`.
- `tree <ruta>`: muestra una estructura de directorios legible.

Ahora abre Text Editor, crea `~/curso-linux/evidencias/sistema.txt` y pega las salidas de `whoami`, `uname -r`, `cat /etc/os-release` e `id`. Los valores siguen viniendo de comandos; no los inventes ni copies valores de otro equipo.

SALIDA REPRESENTATIVA

SALIDA REPRESENTATIVA

```
/home/ubuntu/curso-linux
├── evidencias
│   └── sistema.txt
```

COMPRUEBA.

En Files o con `ls -l ~/curso-linux/evidencias`, verifica que existe `sistema.txt`. En `tree ~/curso-linux` debes reconocer la carpeta `evidencias` y el archivo que acabas de guardar.

Errores frecuentes

[Índice ↑](#)

- Confundir Ubuntu con el kernel Linux.
- Trabajar siempre como `root` en vez de usar `sudo` por comando.
- Ejecutar `apt upgrade` sin revisar qué cambiará.
- Usar `usermod -G` sin `-a` y perder grupos secundarios.

Reto 1 — Inventario reproducible

[Índice ↑](#)[Ver respuesta](#)

Crea `~/curso-linux/evidencias/inventario.txt` desde Text Editor con hostname, distribución, kernel, usuario, grupos y ruta del ejecutable `bash`. Obtén cada valor ejecutando el comando correspondiente; no copies valores de otro equipo. La Clase 3 automatizará este mismo inventario desde terminal.

Criterios de comprobación

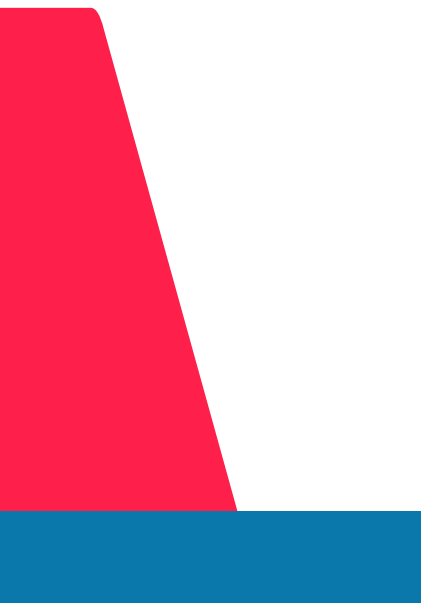
[Índice ↑](#)

- El archivo contiene seis datos con etiquetas legibles.
- Los valores provienen de comandos.
- El comando puede ejecutarse nuevamente sin producir errores.

Checklist

[Índice ↑](#)

- ☐ Distingo kernel y distribución.
- ☐ Distingo terminal, shell y escritorio.
- ☐ Sé consultar `man` y `--help`.
- ☐ Puedo buscar e instalar un paquete.
- ☐ Puedo interpretar UID, GID y grupos.



02

SISTEMA DE ARCHIVOS

Estructura, enlaces y permisos

Tipos de archivo, rutas, jerarquía, montajes y el modelo de acceso para propietario, grupo y otros.

Capítulo 3

FHS

chmod

chown

Índice

[Índice ↑](#)

- [Objetivos](#)
- [Antes de empezar](#)
- [3.1 Tipos de archivo](#)
- [3.2 Enlaces](#)
- [3.3 Rutas o paths](#)
- [3.4 Jerarquía principal](#)
- [3.5 Acceso a sistemas de archivos](#)
- [3.6 Permisos](#)
- [Práctica guiada resuelta](#)
- [Errores frecuentes](#)
- [Reto 3](#)

Objetivos

[Índice ↑](#)

- Reconocer tipos de archivos y rutas.
- Navegar la jerarquía principal de Linux.
- Crear enlaces y explicar su diferencia.
- inspeccionar montajes sin modificar discos.
- aplicar permisos y propietarios de forma consciente.

Antes de empezar

[Índice ↑](#)

Sitúate en la raíz del curso y crea el espacio de trabajo:



TERMINAL · BASH

```
cd ~/linux-desde-cero
mkdir -p laboratorio/enlaces
```

En los ejemplos usaremos estos nombres concretos:

Nombre	Qué representa
config.ini	archivo original
config-duro.ini	enlace duro al original
config-actual.ini	enlace simbólico al original
deploy.sh	archivo para practicar permisos

ANTES DE EJECUTAR.

`laboratorio/enlaces` es una carpeta de práctica de Clase 1. Puedes crearla con `mkdir -p` sin afectar `data/` ni otros ejercicios. Antes de copiar, mover o borrar, confirma la ruta con `pwd` y `ls`.

3.1 Tipos de archivo

[Índice ↑](#)

SITUACIÓN REAL.

Un nombre terminado en `.conf` o `.log` ayuda a una persona, pero Linux decide cómo tratar un objeto por su tipo y permisos. Antes de modificar un archivo recibido, confirma qué es realmente.

El primer carácter de `ls -l` indica el tipo:

Carácter	Tipo	Ejemplo
-	archivo regular	texto, CSV, binario
d	directorio	<code>/home</code>
l	enlace simbólico	acceso alternativo
b / c	dispositivo de bloque/carácter	discos, terminales
s	socket	comunicación local
p	pipe con nombre	comunicación entre procesos

```
file /etc/passwd /bin/ls /dev/null
ls -ld /etc/passwd /tmp /dev/null
```

Las opciones usadas:

- `file < rutas >`: inspecciona contenido y metadatos para estimar el tipo.
- `ls -l`: `-l` muestra permisos, dueño, grupo y tamaño.
- `ls -d`: `-d` describe el directorio indicado, en lugar de listar su contenido.

SALIDA REPRESENTATIVA

```
/etc/passwd: ASCII text
/bin/ls: ELF 64-bit LSB pie executable
/dev/null: character special
drwxrwxrwt ... /tmp
```

`file` inspecciona el contenido o metadatos; la extensión no decide el tipo en Linux.

3.2 Enlaces

[Índice ↑](#)

SITUACIÓN REAL.

Un servicio puede consultar siempre `config-actual`, aunque la configuración real cambie de versión. Un enlace evita copiar el archivo y permite conservar un nombre estable.

Un enlace **no es una segunda copia**. Es otra forma de llegar al mismo contenido, pero hay dos mecanismos que se comportan de forma distinta.

Tipo	Modelo mental	Qué ocurre si el nombre original desaparece	Límite importante
Enlace duro	Dos nombres apuntan al mismo inode y a los mismos datos.	El otro nombre sigue funcionando.	Sólo vive en el mismo sistema de archivos; normalmente no se crea para directorios.
Enlace simbólico	Un archivo pequeño guarda la ruta de otro archivo. Es parecido a un acceso directo.	Queda roto si esa ruta deja de existir.	Puede apuntar a directorios y cruzar sistemas de archivos.

Modelo mental: nombre, inode y contenido

[Índice ↑](#)

Un directorio guarda **nombres**. Cada nombre de archivo apunta a un **inode**, la ficha que identifica los datos y sus metadatos en el sistema de archivos. Por eso dos enlaces duros muestran el mismo inode: son dos nombres para los mismos datos. Un enlace simbólico tiene su propio inode y guarda una ruta hacia otro nombre.

SALIDA REPRESENTATIVA

```
config.ini ─┐
config-duro.ini ─┬─> inode 12345 ─> contenido: version=1
config-actual.ini ─┘
                  ─> guarda la ruta "config.ini"
```

No confundas un enlace duro con un respaldo: ambos nombres comparten los mismos datos. Si se modifica el contenido desde uno, cambia para el otro.

SINTAXIS GENERAL

TERMINAL · BASH

```
ln <archivo_existente> <nuevo_enlace_duro>
ln -s <ruta_objetivo> <nuevo_enlace_simbolico>
```

Ejemplo guiado

[Índice ↑](#)

Trabaja en una carpeta propia para no mezclar este ejemplo con prácticas anteriores. Si ya existe `laboratorio/enlaces/demostracion/`, no repitas la creación de enlaces sobre los mismos nombres: continúa en la comprobación o crea otra carpeta de práctica.

```
mkdir -p laboratorio/enlaces/demostracion
touch laboratorio/enlaces/demostracion/config.ini
```

Abre `config.ini` con Text Editor y escribe `version=1`. Así podrás comprobar que los tres nombres llevan al mismo contenido sin introducir redirecciones en esta clase.

```
ln laboratorio/enlaces/demostracion/config.ini laboratorio/enlaces/demostracion/config-duro.ini
ln -s config.ini laboratorio/enlaces/demostracion/config-actual.ini
ls -li laboratorio/enlaces/demostracion
```

Las opciones usadas:

- `mkdir -p <ruta>`: crea la carpeta de práctica sin fallar si ya existe.
- `touch <archivo>`: crea el archivo vacío si falta; no borra contenido existente.
- `ln <origen> <destino>`: crea un enlace duro al mismo inode.
- `ln -s <objetivo> <enlace>`: `-s` crea un enlace simbólico que guarda una ruta.
- `ls -l`: muestra tipo, permisos y el destino de un enlace simbólico; `ls -i` agrega el inode.

SALIDA REPRESENTATIVA

```
12345 -rw-r--r-- ... config-duro.ini
12345 -rw-r--r-- ... config.ini
12346 lrwxrwxrwx ... config-actual.ini -> config.ini
```

Identifica dos cosas: `config.ini` y `config-duro.ini` comparten `12345`, mientras que `config-actual.ini` empieza con `1` y muestra `-> config.ini`. Los números cambiarán en cada equipo; lo importante es qué nombres comparten número.

Comprueba qué se rompe y qué permanece

[Índice ↑](#)

Abre `config-duro.ini` y `config-actual.ini` con Text Editor: ambos llevan al mismo contenido. Después, sólo dentro de `laboratorio/enlaces/demostracion/`, renombra el archivo original y vuelve a listar:

TERMINAL · BASH

```
mv laboratorio/enlaces/demostracion/config.ini laboratorio/enlaces/demostracion/config-v1.ini
ls -li laboratorio/enlaces/demostracion
```

El enlace duro sigue dando acceso a los datos, porque todavía apunta al inode `12345`. El enlace simbólico queda roto porque aún guarda la ruta `config.ini`. Para repararlo, elimina **sólo el enlace simbólico de práctica** y créalo con la nueva ruta:

TERMINAL · BASH

```
rm laboratorio/enlaces/demostracion/config-actual.ini
ln -s config-v1.ini laboratorio/enlaces/demostracion/config-actual.ini
ls -li laboratorio/enlaces/demostracion
```

CUIDADO.

Ejecuta el `rm` anterior sólo si la ruta completa es `laboratorio/enlaces/demostracion/config-actual.ini`. Nunca practiques este comportamiento con archivos de `/etc`, proyectos reales ni directorios del sistema.

3.3 Rutas o paths

[Índice ↑](#)

SITUACIÓN REAL.

La mayoría de los errores de copia, permisos o borrado empiezan con una ruta mal entendida. Antes de operar, debes saber dónde estás y cómo se interpreta el destino.

- Absoluta: comienza en `/`, por ejemplo `/var/log`.
- Relativa: comienza en el directorio actual, por ejemplo `../data`.
- `.`: directorio actual.
- `..`: directorio padre.
- `~`: home del usuario.

```
pwd
realpath laboratorio/enlaces/config.ini
basename /var/log/syslog
dirname /var/log/syslog
```

Las piezas que debes reconocer:

- `pwd`: muestra la ruta actual.
- `realpath <ruta>`: resuelve una ruta a su forma absoluta.
- `basename <ruta>`: devuelve sólo el último nombre.
- `dirname <ruta>`: devuelve el directorio padre.

3.4 Jerarquía principal

[Índice ↑](#)

SITUACIÓN REAL.

Un operador no busca una configuración de servicio en cualquier carpeta: sabe que `/etc` suele guardar configuración, `/var` datos que cambian y `/home` archivos de usuarios. La jerarquía acelera el diagnóstico.

Ruta	Propósito práctico
<code>/home</code>	datos y configuración de usuarios
<code>/etc</code>	configuración del sistema y servicios
<code>/var</code>	datos variables, cachés y logs
<code>/tmp</code>	temporales; no asumir persistencia
<code>/usr</code>	programas, bibliotecas y datos compartidos
<code>/dev</code>	dispositivos
<code>/proc</code>	vista del kernel y procesos
<code>/run</code>	estado desde el arranque

TERMINAL · BASH

```
ls -lah /  
head -n 5 /proc/meminfo
```

Las opciones usadas:

- `ls -l`: muestra detalles; `-a` incluye nombres ocultos; `-h` hace legibles tamaños.
- `head -n 5 <archivo>`: `-n 5` limita la salida a cinco líneas.

SALIDA REPRESENTATIVA

SALIDA REPRESENTATIVA

```
drwxr-xr-x ... home  
drwxr-xr-x ... etc  
drwxr-xr-x ... var  
MemTotal:    ... kB  
MemAvailable: ... kB
```

3.5 Acceso a sistemas de archivos

[Índice ↑](#)

SITUACIÓN REAL.

En soporte o DevOps alguien puede decir “el disco `/dev/nvme0n1p1` está lleno”, mientras que una aplicación sólo conoce rutas como `/var/log` o `/home`. Un montaje es la relación entre ambos mundos: conecta un dispositivo o sistema de archivos con una ruta dentro del único árbol de Linux.

Antes de continuar

[Índice ↑](#)

Este apartado pertenece a la Clase 2. Trabaja desde la raíz del curso, aunque conserves carpetas de la Clase 1 en `laboratorio/`:

```
cd ~/linux-desde-cero
pwd
```

No necesitas crear discos ni particiones para este ejercicio. Los comandos consultan el estado actual del equipo. Si `laboratorio/` no existe, podrás crearlo más adelante con `mkdir -p laboratorio`; no recales a mano archivos versionados de `data/`.

Modelo mental

[Índice ↑](#)

Piensa en un montaje como una puerta: el **origen** es el dispositivo o sistema de archivos, el **destino** es la carpeta por la que lo usas y el **tipo** indica cómo organiza sus datos. No trabajes con letras de unidad como en otros sistemas; pregunta siempre “¿en qué ruta está montado?”.

En este curso se inspeccionan montajes; no se formatean discos ni se modifican particiones.

Sintaxis y ejemplo guiado

[Índice ↑](#)

```
lsblk -f          # dispositivos, tipo y punto de montaje
findmnt <ruta>    # qué origen está montado en una ruta
df -hT <ruta>     # capacidad del sistema de archivos de esa ruta
```

```
lsblk -f
findmnt /
df -hT /
```


SALIDA REPRESENTATIVA

SALIDA REPRESENTATIVA

```
TARGET SOURCE  FSTYPE OPTIONS
/           /dev/...  ext4    rw,relatime
```

Lee la salida en este orden:

1. `lsblk -f` muestra discos y particiones. Busca las columnas `FSTYPE` y `MOUNTPOINTS`; no cambies nada desde esa pantalla.
2. `findmnt /` responde qué está montado exactamente en la raíz `/`.
3. `df -hT /` responde cuánto espacio queda para las rutas que viven bajo esa raíz.

COMPRUEBA.

Si una aplicación guarda logs en `/var/log`, prueba `findmnt /var/log` y `df -hT /var/log`. Puede pertenecer al mismo montaje que `/` o a uno independiente; la salida, no una suposición, te da la respuesta.

- `lsblk -f`: dispositivos, sistema de archivos, etiquetas y puntos de montaje.
- `findmnt`: relación entre destino y origen.
- `df -hT`: espacio disponible y tipo; `-h` usa unidades legibles y `-T` agrega el tipo.
- `ext4` es común en Ubuntu; XFS y Btrfs ofrecen decisiones diferentes, pero el usuario sigue trabajando mediante rutas.

CUIDADO.

Comandos como `mkfs`, `fdisk`, `parted` o `mount` pueden cambiar almacenamiento. No son parte de esta práctica. Primero identifica origen, destino y consecuencia antes de ejecutar una operación sobre discos.

3.6 Permisos

Índice ↑

SITUACIÓN REAL.

Un archivo de configuración no debe ejecutarse y un script no debe ser modificable por cualquiera. Los permisos expresan esa intención; 777 no es una solución universal.

SALIDA REPRESENTATIVA

```
-rwxr-x--- 1 alumno devops 1200 jul 4 10:00 deploy.sh
├───┬───┬───    dueño grupo
│   │   │       └─── otros: ---
│   │   └───     grupo: r-x
│   └───        dueño: rwx
└───            archivo regular
```

Permiso	Archivo	Directorio	Valor
r	leer contenido	listar nombres	4
w	modificar	crear/eliminar entradas	2
x	ejecutar	atravesar/acceder	1

TERMINAL · BASH

```
touch laboratorio/enlaces/deploy.sh
chmod u=rwx,g=rx,o= laboratorio/enlaces/deploy.sh
chmod 750 laboratorio/enlaces/deploy.sh
ls -l laboratorio/enlaces/deploy.sh
```

Las opciones y modos usados:

- `chmod u=rwx,g=rx,o=` : asigna permisos simbólicos para dueño, grupo y otros.
- `chmod 750` : forma octal de `rwxr-x---`.
- `ls -l` : permite comprobar el modo antes y después.

`chown` y `chgrp` cambian dueño o grupo y se usan sólo cuando el responsable de la ruta lo requiere. Los practicaremos con una VM autorizada; no copies nombres de usuario de otro equipo.

SALIDA REPRESENTATIVA

SALIDA REPRESENTATIVA

```
-rwxr-x--- ... usuario grupo ... laboratorio/enlaces/deploy.sh
```

- **chmod**: cambia bits de permiso.

- `chown usuario:grupo` : cambia dueño y grupo cuando existe una necesidad autorizada.
- `chgrp` : cambia sólo el grupo.
- `750` equivale a `rxr-x---` .

Práctica guiada resuelta

[Índice ↑](#)

La práctica crea este árbol, por lo que no necesitas preparar los archivos a mano:

```
laboratorio/proyecto/  
├─ bin/status.sh  
├─ config/app.env  
└─ logs/
```

```
mkdir -p laboratorio/proyecto/{config,logs,bin}  
touch laboratorio/proyecto/config/app.env  
touch laboratorio/proyecto/bin/status.sh  
chmod 640 laboratorio/proyecto/config/app.env  
chmod 750 laboratorio/proyecto/bin/status.sh  
ln -s ../config/app.env laboratorio/proyecto/bin/config-actual  
ls -l laboratorio/proyecto/config laboratorio/proyecto/bin
```

Abre `status.sh` con Text Editor y escribe `echo "servicio listo"`. Guarda antes de intentar ejecutarlo. Esta primera clase evita redirecciones; en Clase 3 aprenderás a crear ese contenido desde terminal.

Salida principal:

```
-rwxr-x--- ... status.sh  
lrwxrwxrwx ... config-actual -> ../config/app.env  
-rw-r----- ... app.env
```

El enlace muestra `rwX` por sí mismo; el acceso efectivo depende del objetivo y de los directorios de la ruta.

Para ejecutar el script después de guardarlo, usa `laboratorio/proyecto/bin/status.sh`. Si ves `Permission denied`, confirma primero `pwd` y luego ejecuta `ls -l laboratorio/proyecto/bin/status.sh`.

Errores frecuentes

[Índice ↑](#)

- Aplicar `chmod -R 777` para ocultar un problema.
- Confundir espacio de `df` con tamaño de archivos calculado por `du`.
- Crear un enlace simbólico relativo desde el directorio equivocado.
- Ejecutar `chown` o `rm` sin comprobar la ruta.

Reto 3 — Directorio compartido

[Índice ↑](#)

Ver respuesta

Crea `laboratorio/compartido` para un equipo: dueño con control total, grupo con lectura/escritura/acceso y otros sin acceso. Dentro crea `app.env` como configuración no ejecutable, `check.sh` como script ejecutable y `check-actual` como enlace simbólico al script.

Usa `touch` para crear los dos archivos y Text Editor para escribir `echo "servicio listo"` dentro de `check.sh`. No uses redirecciones todavía; se explican en la Clase 3.

Criterios de comprobación

[Índice ↑](#)

- Los modos reflejan necesidades distintas para directorio, configuración y script.
- El script se ejecuta y la configuración no.
- `readlink` permite identificar el objetivo del enlace.

Checklist

[Índice ↑](#)

- ☐ Distingo tipo de archivo, extensión e inode.
- ☐ Uso rutas absolutas y relativas.
- ☐ Puedo explicar enlaces duros y simbólicos.
- ☐ Inspecciono montajes sin modificar discos.
- ☐ Interpreto y cambio permisos simbólicos y octales.

03

TRABAJO EN TERMINAL

El shell: comandos esenciales

Resolución de comandos, navegación y operaciones seguras con archivos hasta permisos y propiedad.

Capítulo 7.1–7.15

bash

cp · mv · rm

file · stat

Índice

[Índice ↑](#)

- [Objetivos](#)
- [Antes de empezar](#)
- [7.1–7.3 Shell, comandos y directorio personal](#)
- [7.4 Listar](#)
- [7.5–7.8 Directorios y navegación](#)
- [7.9 Unidades y montajes](#)
- [7.10 Copiar](#)
- [7.11 Mover y renombrar](#)
- [7.12 Enlaces](#)
- [7.13 Borrar](#)
- [7.14 Identificar](#)
- [7.15 Permisos y propiedad](#)
- [Ruta práctica de la Clase 2](#)
- [7.16 Espacio](#)
- [7.17–7.18 Visualizar](#)
- [7.19 Buscar](#)
- [7.20 Empaquetar y comprimir](#)
- [7.21 Impresión](#)
- [Práctica guiada: respaldo de evidencia](#)
- [Errores frecuentes](#)
- [Reto 7](#)

Objetivos

[Índice ↑](#)

- Navegar y administrar archivos desde Bash.
- Consultar ayuda y verificar el efecto de cada comando.
- Buscar, medir, empaquetar y recuperar información.
- Aplicar opciones de seguridad antes de borrar o sobrescribir.

Antes de empezar

[Índice ↑](#)

Este capítulo forma una secuencia: cada apartado reutiliza archivos del anterior. Al iniciar la **Clase 1**, empieza desde la raíz del curso y prepara una copia limpia de los datos:

```
cd ~/linux-desde-cero
bash ejercicios-bash-scripting/preparar-lab.sh
mkdir -p laboratorio/shell/{entrada,salida,backup}
```

TERMINAL · BASH

Trabajaremos con `modelo.txt` como archivo original, `backup-modelo.txt` como copia y `modelo-validado.txt` como nombre nuevo. No uses archivos personales para estas prácticas.

`preparar-lab.sh` elimina y recrea `laboratorio/`. Úsalo al iniciar la Clase 1 o cuando el instructor indique un reinicio deliberado. Si ya estás en la Clase 2 y quieres conservar evidencia previa, ve al apartado “Antes de continuar con la Clase 2” y no ejecutes este reinicio.

7.1–7.3 Shell, comandos y directorio personal

[Índice ↑](#)

SITUACIÓN REAL.

La terminal no es un lugar para memorizar hechizos: es una conversación con el sistema. El shell decide qué programa ejecuta y conserva tu contexto, por ejemplo la carpeta actual.

Bash interpreta palabras, expansiones, redirecciones y operadores. Un comando puede ser **builtin del shell** o un **ejecutable externo**:

Tipo	Descripción	Ejemplos
Builtin	Integrado en el propio shell; no existe como archivo en el sistema	<code>cd</code> , <code>echo</code> , <code>export</code> , <code>source</code>
Alias	Atajo definido en la sesión o en <code>.bashrc</code> ; envuelve otro comando	<code>ls</code> → <code>ls --color=auto</code> , <code>grep</code> → <code>grep --color=auto</code>
Función	Función definida en el shell o en archivos de configuración	funciones personalizadas en <code>.bashrc</code>
Externo	Archivo binario en el sistema de archivos; el shell lo localiza vía <code>\$PATH</code>	<code>ls</code> , <code>grep</code> , <code>awk</code> , <code>sed</code> , <code>find</code> , <code>cp</code> , <code>mv</code> , <code>rm</code> , <code>cat</code> , <code>git</code> , <code>python3</code>

El shell resuelve en este orden: alias → función → builtin → externo.

¿Por qué importa? Los builtins son más rápidos (no crean un proceso nuevo) y pueden modificar el estado del shell actual. `cd`, por ejemplo, solo funciona como builtin: si fuera externo, cambiaría el directorio de un proceso hijo y el shell padre nunca lo notaría.

Usa `type` para identificar la naturaleza de cualquier comando:

```

type cd
type ls
type echo

```

La salida depende de tu configuración; ejemplos comunes:

```

cd is a shell builtin
ls is aliased to `ls --color=auto`
echo is a shell builtin

```

Para saltar alias y llegar al ejecutable real, usa `type -a` o `which`:

TERMINAL · BASH

```
type -a ls      # alias → /usr/bin/ls → /bin/ls  (alias + externo)
type -a grep    # alias → /usr/bin/grep          (alias + externo)
type -a echo    # builtin → /usr/bin/echo        (builtin Y también externo)
type -a mkdir   # solo /usr/bin/mkdir            (puro externo, sin alias ni builtin)
type -a cd      # cd is a shell builtin          (solo builtin, sin archivo)

which ls        # /usr/bin/ls  (ignora el alias, devuelve el binario)
which grep      # /usr/bin/grep (ignora el alias, devuelve el binario)
which echo      # /usr/bin/echo (ignora el builtin, devuelve el binario externo)
which mkdir     # /usr/bin/mkdir (externo puro, sin alias ni builtin)
which cd        # (sin salida – cd es builtin, no tiene archivo ejecutable)
```

Las opciones usadas:

- `type -a <nombre>`: muestra todas las formas en que el shell puede resolver un nombre.
- `which <nombre>`: busca un ejecutable en `PATH`; úsalo como pista, no para sustituir a `type` al investigar alias o builtins.

Directorio personal y navegación básica:

TERMINAL · BASH

```
echo "$HOME"
cd
pwd
```

SALIDA REPRESENTATIVA

SALIDA REPRESENTATIVA

```
/home/ubuntu
/home/ubuntu
```

7.4 Listar: `ls`

[Índice ↑](#)

SITUACIÓN REAL.

Antes de copiar, mover o borrar, lista la ruta exacta. `ls` responde qué hay y, con opciones, quién es dueño, cuánto mide y cuándo cambió.

TERMINAL · BASH

```
ls -lah
ls -lt laboratorio
```

- `-l` : formato largo.
- `-a` : incluye nombres que comienzan con `.`.
- `-h` : tamaños legibles junto con `-l`.
- `-t` : ordena por modificación.

SALIDA REPRESENTATIVA

SALIDA REPRESENTATIVA

```
drwxr-xr-x ... usuario grupo ... laboratorio
-rw-r--r-- ... usuario grupo ... archivo.txt
```

No analices `ls` en scripts complejos: nombres con espacios o saltos de línea requieren herramientas como `find`.

7.5–7.8 Directorios y navegación

[Índice ↑](#)

SITUACIÓN REAL.

Una operación segura empieza por la carpeta correcta. `cd` cambia el contexto de la terminal; por eso siempre confirmas con `pwd` antes de actuar.

TERMINAL · BASH

```
mkdir -p laboratorio/shell/{entrada,salida,backup}  
cd laboratorio/shell/entrada  
pwd  
cd -  
rmdir laboratorio/shell/backup
```

Después de `cd laboratorio/shell/entrada`, `pwd` debe terminar en `/laboratorio/shell/entrada`. `cd -` vuelve a la raíz del curso desde la que comenzaste. `rmdir` elimina `backup` porque todavía está vacío; más adelante los respaldos se crearán con otros nombres.

- `mkdir -p`: crea padres y tolera rutas existentes.
- `rmdir`: elimina sólo directorios vacíos.
- `cd -`: regresa a la ruta anterior.
- `pwd`: muestra la ruta efectiva actual.

Salida representativa después de `cd laboratorio/shell/entrada`:

SALIDA REPRESENTATIVA

```
/home/usuario/linux-desde-cero/laboratorio/shell/entrada
```

7.9 Unidades y montajes

[Índice ↑](#)

Linux presenta los sistemas de archivos dentro de una sola jerarquía:



TERMINAL · BASH

```
lsblk -f  
findmnt /
```

- `lsblk -f`: `-f` agrega tipo de sistema de archivos, etiqueta y punto de montaje.
- `findmnt /`: pregunta qué está montado en la ruta raíz.

No accedas a “una unidad” por letra; identifica su punto de montaje.

7.10 Copiar: `cp`

[Índice ↑](#)

SITUACIÓN REAL.

Antes de cambiar una configuración o un artefacto, haces una copia. La regla es siempre la misma: primer argumento = origen, segundo argumento = destino.

SINTAXIS GENERAL

```
cp [opciones] <origen> <destino>
```

En el ejemplo resuelto, `modelo.txt` es el origen. Créalo vacío con `touch`; si quieres contenido, ábrelo con Text Editor y escribe `accuracy=0.93`.

```
touch laboratorio/shell/entrada/modelo.txt
cp -v laboratorio/shell/entrada/modelo.txt laboratorio/shell/backup-modelo.txt
cp -a laboratorio/shell/entrada laboratorio/shell/entrada-copia
```

- `-v`: informa operaciones.
- `-a`: copia recursivamente preservando metadatos apropiados.
- Para evitar sobrescritura accidental puede usarse `cp -i`.

Salida representativa de `cp -v`:

```
'laboratorio/shell/entrada/modelo.txt' -> 'laboratorio/shell/backup-modelo.txt'
```

7.11 Mover y renombrar: `mv`

[Índice ↑](#)

SITUACIÓN REAL.

Renombrar una copia deja claro cuál configuración o versión es la vigente sin cambiar su contenido.

Sintaxis general: `mv <origen> <destino>`. Aquí renombraremos la copia `backup-modelo.txt` como `modelo-validado.txt`; el archivo original `entrada/modelo.txt` permanece intacto.



TERMINAL · BASH

```
mv -i laboratorio/shell/backup-modelo.txt laboratorio/shell/modelo-validado.txt
```

Dentro del mismo sistema de archivos, renombrar suele ser inmediato. Entre sistemas puede implicar copiar y eliminar.

- `-i`: solicita confirmación antes de sobrescribir un destino existente.

7.12 Enlaces: ln

[Índice ↑](#)

SITUACIÓN REAL.

Un enlace simbólico permite usar un nombre estable como `modelo-actual` sin duplicar el archivo. Es una referencia, no una segunda copia.

La diferencia entre enlace duro, enlace simbólico e inode se trabaja paso a paso en la sección 3.2. Aquí sólo aplicas un enlace simbólico relativo dentro del laboratorio del shell.

El objetivo se escribe como `entrada/modelo.txt` porque el enlace estará dentro de `laboratorio/shell`. Esa ruta se interpreta desde la ubicación del enlace, no desde tu terminal.

TERMINAL · BASH

```
ln -s entrada/modelo.txt laboratorio/shell/modelo-actual
readlink laboratorio/shell/modelo-actual
```

`-s` crea un enlace simbólico; `readlink` muestra su objetivo almacenado.

SALIDA REPRESENTATIVA

SALIDA REPRESENTATIVA

```
entrada/modelo.txt
```

7.13 Borrar: `rm`

[Índice ↑](#)

CUIDADO.

Borrar es la única operación de este bloque que no tiene recuperación general en terminal. Por eso se practica sólo dentro de `laboratorio/shell/` y con confirmaciones.

```
rm -i laboratorio/shell/modelo-validado.txt
rm -rI laboratorio/shell/entrada-copia
```

- `-i`: pregunta por cada archivo.
- `-r`: recorre directorios.
- `-I`: una confirmación para una eliminación recursiva o numerosa.

Antes de un `rm -r`, ejecuta `ls` sobre la misma ruta. No uses `-f` como solución automática.

Ambos comandos pedirán confirmación. Responde `y` sólo después de comprobar que las rutas comienzan con `laboratorio/shell/`.

Salida representativa de una confirmación:

```
rm: remove regular file 'laboratorio/shell/modelo-validado.txt'?
```

7.14 Identificar: `file` y `stat`

[Índice ↑](#)

SITUACIÓN REAL.

Si un archivo no se comporta como esperas, primero identifica su tipo, tamaño y permisos. La extensión no basta para diagnosticarlo.

TERMINAL · BASH

```
file laboratorio/shell/entrada/modelo.txt
stat -c '%F | %s bytes | %a | %n' laboratorio/shell/entrada/modelo.txt
```

SALIDA REPRESENTATIVA

SALIDA REPRESENTATIVA

```
ASCII text
regular file | 14 bytes | 644 | laboratorio/shell/entrada/modelo.txt
```

- `%F`: tipo.
- `%s`: bytes.
- `%a`: modo octal.
- `%n`: nombre.

Las opciones usadas:

- `stat -c '<formato>'`: `-c` define qué campos mostrar.
- `%F`, `%s`, `%a` y `%n`: tipo, bytes, permisos octales y nombre.

7.15 Permisos y propiedad

[Índice ↑](#)

SITUACIÓN REAL.

Los permisos expresan quién puede leer, modificar o ejecutar. Cambiarlos debe ser una decisión basada en el uso del archivo, no un intento de hacer desaparecer un error.

TERMINAL · BASH

```
chmod 640 laboratorio/shell/entrada/modelo.txt  
ls -l laboratorio/shell/entrada/modelo.txt
```

- `chmod` : modo.
- `chmod 640` : dueño con lectura/escritura, grupo con lectura y otros sin acceso.
- `ls -l` : confirma el modo aplicado.

`chgrp` y `chown` cambian grupo o dueño y se usan sólo con autorización sobre la ruta. En esta primera clase se observan y se explican; no copies un nombre de usuario o grupo de otra máquina.

SALIDA REPRESENTATIVA

SALIDA REPRESENTATIVA

```
-rw-r----- ... usuario grupo ... laboratorio/shell/entrada/modelo.txt
```

Antes de continuar con la Clase 2

[Índice ↑](#)

Esta sesión empieza **después** de navegación, archivos y permisos. Por eso puedes encontrar trabajo de la Clase 1 dentro de `laboratorio/`. No hace falta borrarlo ni volver a crearlo para usar esta guía.

ANTES DE EJECUTAR.

Sitúate en la raíz del curso y comprueba los dos archivos de lectura de esta clase. Son fixtures del repositorio: si faltan, no los crees vacíos; recupera el repositorio o pide apoyo al instructor.

TERMINAL · BASH

```
cd ~/linux-desde-cero
pwd
ls data/dummy_logs.txt data/specials.txt
```

El directorio `laboratorio/` sí es tu área de práctica. Puedes crear una carpeta nueva sin tocar los ejercicios anteriores:

TERMINAL · BASH

```
mkdir -p laboratorio/shell
```

Si conservas `laboratorio/shell/entrada/modelo.txt` o `laboratorio/proyecto/` de la Clase 1, úsalos como evidencia de tu avance. Esta clase no depende de ellos. **No ejecutes `preparar-lab.sh` sólo para empezar la Clase 2:** ese script reinicia `laboratorio/` y elimina evidencia previa.

Ruta práctica de la Clase 2

[Índice ↑](#)

La numeración del temario conserva `du` y `df` primero, pero en clase conviene aprender en este orden:

1. leer un archivo sin modificarlo (`head` , `tail` , `less`);
2. buscar una pista o un archivo (`grep` , `find`);
3. decidir si falta espacio (`du` , `df`);
4. crear y comprobar un respaldo (`tar` , `gzip` , `sha256sum`);
5. relacionar archivos, montajes y escritorio gráfico.

Los mismos comandos sirven en distintos roles. Un técnico de soporte busca un archivo de un usuario; alguien de backend revisa configuración y logs antes de desplegar; una persona de operaciones investiga un incidente sin modificar el servidor.



EVIDENCIA Y CIERRE

Clase 1 completada

Al terminar la Clase 1, conserva una evidencia reproducible de tu sistema y de la estructura creada.



Identifico distribución, kernel, usuario, grupos y shell activo.



Navego y opero archivos con rutas, origen y destino claros.



Interpreto enlaces, propietario, grupo y permisos antes de modificarlos.

SIGUIENTE: CLASE 2 • SHELL ÚTIL Y ESCRITORIOS